LLM Security Guard for Code

Arya Kavian
Department of Electrical and Computer Engineering, University of Science and Technology of Mazandaran, Behshahr, Iran

Mohammad Mehdi Pourhashem Kallehbasti
Department of Electrical and Computer Engineering, University of Science and Technology of Mazandaran, Behshahr, Iran

Sajjad Kazemi
Department of Electrical and Computer Engineering, University of Science and Technology of Mazandaran, Behshahr, Iran

Ehsan Firouzi
Technische Universität Clausthal
Germany

Mohammad Ghafari
Technische Universität Clausthal
Germany

ABSTRACT

Many developers rely on Large Language Models (LLMs) to facilitate software development. Nevertheless, these models have exhibited limited capabilities in the security domain. We introduce LLMSecGuard, a framework to offer enhanced code security through the synergy between static code analyzers and LLMs. LLMSecGuard is open source and aims to equip developers with code solutions that are more secure than the code initially generated by LLMs. This framework also has a benchmarking feature, aimed at providing insights into the evolving security attributes of these models.

CCS CONCEPTS

• **Security and privacy → Software security engineering**.

KEYWORDS

Security analysis, secure code generation, code models

ACM Reference Format:
Arya Kavian, Mohammad Mehdi Pourhashem Kallehbasti, Sajjad Kazemi, Ehsan Firouzi, and Mohammad Ghafari. 2024. LLM Security Guard for Code. In *28th International Conference on Evaluation and Assessment in Software Engineering (EASE 2024), June 18–21, 2024, Salerno, Italy*. ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3661167.3661263

1 INTRODUCTION

The use of Large Language Models (LLMs) such as ChatGPT and Copilot has become popular for software development activities such as coding, design, comprehension, etc. [7, 17]. Nevertheless, hallucination, i.e., "presenting incorrect information as if it is correct", poses serious challenges for LLM users [18]. This issue is more prevalent in domains where reliable training content is scarce, such as in the realm of code security. Indeed, recent studies have shown that code models are widely adopted for code generation [12], but they have limited capabilities in the software security domain [2, 25].

This work is licensed under a Creative Commons Attribution International 4.0 License.

EASE 2024, June 18–21, 2024, Salerno, Italy
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1701-7/24/06
https://doi.org/10.1145/3661167.3661263

Therefore, a vulnerable code piece that an LLM mistakenly recommends as a secure solution could compromise the entire system's security if it is used without enough scrutiny.

We introduce LLMSecGuard, a framework specifically designed to examine the security properties of LLMs for code generation and harness their analytical capabilities to facilitate secure code development. It applies static security analysis on LLM-generated code to uncover potential security issues and guides LLMs in resolving such issues in the code. Moreover, LLMSecGuard can assess the security properties of LLMs and benchmark them across different CWEs. In summary, LLMSecGuard contributes to more secure software development. Unlike existing code assistants, developers can integrate unlimited LLMs and code analysis engines into this framework through REST APIs. LLMSecGuard is open-source and publicly available on GitHub.[1]

The remainder of this paper is structured as follows. In Section 2, we motivate this work. In Section 3, we introduce LLMSecGuard, and in Section 4, we explain its two use cases. In Section 5, we present related work. In Section 6, we outline our plans, and in Section 7, we conclude this paper.

2 MOTIVATION

Security issues are pervasive in multiple domains [4], from mobile applications [9, 11] and web servers [10], to critical software systems [27]. There are program analysis tools designed to uncover security issues, but studies indicate their limitations [5, 6], as well as their lack of popularity among developers [14]. Unfortunately, the security landscape may not improve as we witness the popularity of language models for code generation [12]. It is concerning that developers place undue trust in these models, which are known to generate insecure code examples [2, 25].

In the following, we explain how we could iteratively ask ChatGPT 3.5 to generate secure code examples for symmetric encryption. Encryption plays a crucial role in our digital world, but its correct adoption has proven to be challenging for developers [15]. For instance, the analysis of 489 open-source Java projects showed that 85% of cryptography APIs are misused [16]. Importantly, existing tools that identify and fix such issues do not fulfill developers' expectations [1, 28].

[1] https://github.com/aryakvnust/LLMSecGuard

```
1  public class SecureEncryptionExample {
2      public static void main(String[] args) throws Exception {
3          String username = "bob@google.org";
4          String password = "Password1";
5          String secretID = "BlahBlahBlah";
6          String SALT2 = "deliciously salty";
7          // Generate a secure random IV (Initialization Vector)
8          SecureRandom secureRandom = new SecureRandom();
9          byte[] iv = new byte[12];
10         secureRandom.nextBytes(iv);
11         // Derive a key from the password using a secure method
12         byte[] key = deriveKey(SALT2, username, password);
13         // Create a SecretKey object using the derived key
14         SecretKey secretKey = new SecretKeySpec(key, "AES");
15         // Instantiate the cipher with AES in GCM mode
16         Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
17         ...
18     }
19     private static byte[] deriveKey(String salt, String username,
         String password) throws Exception {
20         // Concatenate salt, username, and password
21         String combined = salt + username + password;
22         // Use a secure hash function (SHA-256) to derive a key
23         MessageDigest md = MessageDigest.getInstance("SHA-256");
24         byte[] hash = md.digest(combined.getBytes());
25         return Arrays.copyOf(hash, 16);
26     }
27 }
```

Listing 1: ChatGPT's improved response to Post ID 3451670

Consider the StackOverflow post ID 3451670, which suffers from security issues such as "ECB encryption mode", "static or constant key", "static salt for key derivation", and "hardcoded password". When we presented this inquiry to ChatGPT, requesting a "secure Java code example", the resulting response still contained identical security issues.

We then explicitly instructed ChatGPT that *"I expect a secure encryption algorithm, a secure encryption mode, and a secure Initialization Vector (IV). Do not use hard-coded secrets, suggest best practices, and make sure that your example provides confidentiality and integrity"*. Listing 1 presents the response.[2]

ChatGPT identified the constant key, and it correctly suggested the secure "GCM" encryption mode as a best practice. However, violations such as "static salt for key generation" and "hardcoded password" were in the response without any proper warning. The suggested code, at line 23, included the "SHA-256" hash function for key generation, which is not a best practice. Anyway, when we asked about the security risk at line 6 (ie., static salt), ChatGPT responded: *"Using a hard-coded string as a salt is generally not considered a good practice for security. Ideally, salt should be unique for each user and should be randomly generated for each password. Using a constant salt for all users could make the system more vulnerable to certain types of attacks, such as rainbow table attacks"*. It then provided the code example in Listing 2 as a replacement.

```
// Generate a secure random salt
SecureRandom secureRandom = new SecureRandom();
byte[] salt = new byte[16];
secureRandom.nextBytes(salt);
// Convert the salt to a Base64-encoded string for storage
String SALT2 = Base64.getEncoder().encodeToString(salt);
```

Listing 2: ChatGPT's suggestion to replace a static salt

[2]https://chat.openai.com/share/8ce9eb6e-88b7-44dc-807a-51d04e7d446c

Subsequently, we asked, *"Are there any other security risks in your provided code?"*. It failed to detect the hardcoded password, but at this stage, it correctly suggested a key derivation function (KDF) instead of the hash function for generating a password-based key. Upon a direct inquiry about the line where a hardcoded password existed, it offered a secure suggestion.

Through proper interactions with ChatGPT, specifically by listing potential security issues and their locations within the code, it is feasible to harness ChatGPT's power to generate secure code examples. However, novice developers may not possess the necessary expertise to provide such inputs. Hence, we introduce LLMSecGuard, a framework that links LLMs and static analysis tools to overcome this limitation.

3 LLMSecGuard

We introduce LLMSecGuard, an open-source framework to offer enhanced code security through the synergy between code security analyzers and LLMs. The primary objective of LLMSecGuard is (i) to enhance the security of LLM-generated code, and (ii) to benchmark the security properties of LLMs. It adopts a RESTful architecture, implemented in Python using Django and Flask, and uses SQLite for data persistence. Figure 1 illustrates the schematic architecture of LLMSecGuard. We explain the main components in the following.

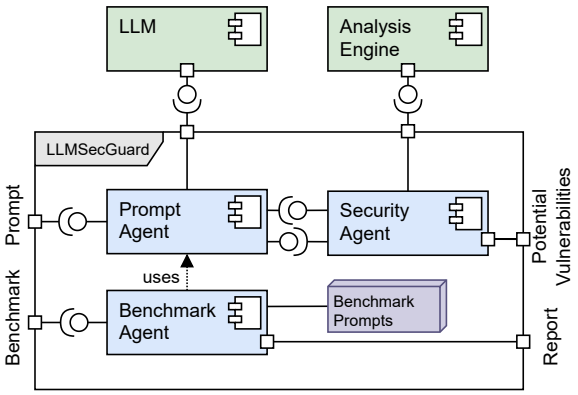

Figure 1: LLMSecGuard's components

Prompt Agent. This component is responsible for receiving a prompt and providing other components with the code model's response. Upon receipt of a prompt, "Prompt Agent" undertakes the task of engineering a response. Particularly, it can reformulate a prompt, pass the prompt to code models, collect the response, and forward the result to other components.

Security Agent. This component has a pivotal role in uncovering security issues in LLM-generated code. Precisely, it is entrusted to pass the code to static code analysis engines (such as Semgrep and Weggli), and to collect potential security vulnerabilities.

Benchmark Agent. This component puts different LLMs to security test. Particularly, it evaluates the security properties of LLMs based on a set of challenges structured in JSON format. Each challenge includes a prompt and the expected outcome, and "Benchmark Agent" is responsible for flagging LLMs that pass the challenge.

4 USAGE SCENARIOS

We describe the LLMSecGuard's main configurations including its connection with external entities such as LLMs and code analyzers. We then explain the interplay between different components through two main usage scenarios, namely "benchmarking" and "code generation".

4.1 Setup

There are several key configurations for running LLMSecGuard that users can adjust according to their preferences.

LLMSecGuard requires a minimum of one LLM and one code analysis engine to operate effectively. This can be easily set up by providing the API information (e.g., API endpoint and API key) associated with each entity. There is no limitation, and users can add as many entities as they wish. Currently, we have instantiated LLMSecGuard with ANYSCALE Llama2 [19], and Weggli and Semgrep static code security analysis tools [22, 26]. Weggli is a robust and efficient semantic code analysis for C and C++ programs, and Semgrep is designed for programs written in languages such as C#, Java, Java, JavaScript, Python, PHP, Ruby, Scala, etc.

LLMSecGuard relies on CyberSecEval, a benchmark specifically designed to evaluate the cybersecurity aspects of LLMs functioning as coding assistants [3]. It comprises exhaustive test cases to assess the potential of LLMs in producing insecure code and facilitating cyberattacks. Nonetheless, users have the flexibility to update this benchmark or replace it with one of their preferences.

LLMSecGuard includes a few execution parameters as well. The primary one is the "termination condition" designed to prevent the system from looping indefinitely. This condition determines how many iterations a code piece should undergo improvement and analysis if potential vulnerabilities persist.

The "benchmark interval" parameter determines how often the benchmark process is executed, with options such as biweekly or monthly intervals. Given that LLMs evolve and their security properties may change, regular updates are necessary to ensure that results remain current.

Lastly, users can choose the LLMs of interest for benchmarking, select specific code analysis engines, and specify challenges that should be considered for measurement.

4.2 Security Benchmarking

In this scenario, a set of existing prompts, referred to as benchmark prompts, undergo evaluation. In particular, the "Benchmark Agent" sends each prompt in the benchmark to the "Prompt Agent". This component dispatches the prompt to every LLM considered for benchmarking, collects responses for each prompt, and forwards them to the "Benchmark Agent". Each prompt presents a challenge, and "Benchmark Agent" determines the extent to which an LLM succeeds in resolving it by comparing the response to the expected outcome. For instance, in terms of vulnerable code, it measures the quantity and severity of potential CWEs. Each LLM is assigned a performance score for each challenge, and ultimately, LLMs are ranked.

At present, LLMSecGuard relies on the CyberSecEval benchmark, which incorporates extensive prompts and regular expressions to uncover potential issues.

4.3 Secure Code Generation

Figure 2 illustrates the workflow in this scenario. It begins with the user providing a prompt of interest, anticipating the output to be a piece of code. The "Prompt Agent" assumes control and forwards the user's prompt to the top LLM determined in the benchmarking scenario (unless the user chooses a different LLM). Upon receiving the LLM's response, this component then transfers the produced code to the "Security Agent" for security inspection. The "Security Agent" collaborates with external analysis engines (configured earlier) to uncover potential vulnerabilities and respective lines in the code. If the termination condition is met (i.e., there is no vulnerability or the maximum analysis iterations is reached), the code, along with its vulnerability information, is immediately reported back to the user. In cases where the termination condition is not met, this information is relayed to the "Prompt Agent". This component formulates a new prompt based on the collected vulnerability information and queries the LLM once again.

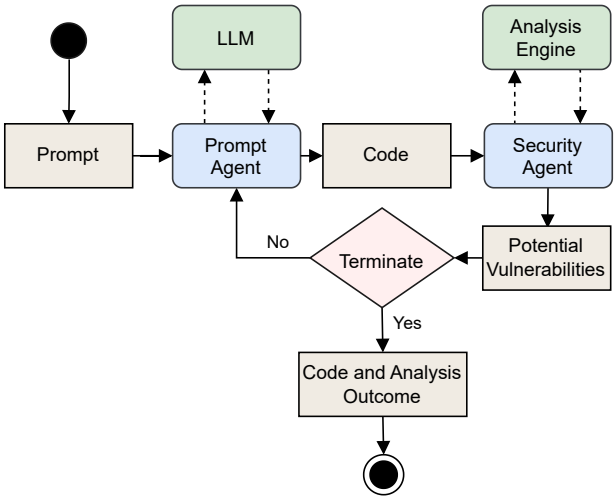

Figure 2: LLMSecGuard's secure code generation workflow

5 RELATE WORK

The popularity of AI-generated code has attracted the attention of security researchers. Pearce et al. assessed Copilot's performance in suggesting code across 89 scenarios aligned with MITRE's "Top 25" CWEs, revealing that approximately 40% of the generated programs contained vulnerabilities [21]. Nonetheless, in a later study, Asare et al. came to a different conclusion [2]. Precisely, Copilot did not consistently reproduce past vulnerabilities introduced by humans. In about 25% of cases, Copilot even proposed fixes for these vulnerabilities, suggesting a potentially lower susceptibility to security risks compared to human programmers. Fu et al. analyzed 435 code snippets generated by Copilot in public GitHub projects and found that 35% of the snippets exhibit CWEs [8]. Mousavi et al. designed 48 programming tasks for five common security APIs in Java and evaluated the performance of ChatGPT in developing these tasks. They uncovered a concerning level of security, specifically, they found an average misuse rate of 70% in these tasks [20].

Researchers have also developed benchmarks for assessing the security of LLM-generated code. Bhatt et al. developed CYBERSECEVAL, a benchmark tailored to evaluate the cybersecurity risks posed by LLMs [3]. Hajipour et al. examined code models for generating high-risk security weaknesses and built a collection of diverse non-secure prompts for various vulnerability scenarios, which can be used as a benchmark to compare security weaknesses in LLMs [13].

6 FUTURE WORK

We plan to investigate whether LLMSecGuard will effectively support developers in writing secure code in real-world scenarios. Two groups of developers, both utilizing LLMs for coding, will be recruited for our study. We will assign the same programming tasks to each group, instructing one to utilize LLMs freely and the other to exclusively use LLMSecGuard during the coding process. We will measure the time taken to complete each task, the number, and the severity of vulnerabilities. We will compare the results across two groups as well as based on participants' experience.

Pending positive evaluations, our ultimate goal is to integrate LLMSecGuard into at least one popular IDE (Integrated Development Environment), as deemed essential by developers for a seamless user experience [24]. Furthermore, this integration would allow LLMSecGuard to gather development context, thereby enabling the formulation of better prompts for code generation.

Future work could also investigate code changes in each iteration between "Prompt Agent" and "Security Agent", as well as examine how the engineering of prompts and vulnerability information affects the performance of LLMs.

Finally, it is noteworthy that although a code snippet may not inherently contain vulnerabilities, its integration into a program or execution within a specific environment could still pose security risks and therefore warrants investigation [23].

7 CONCLUSION

We introduced LLMSecGuard, an open-source framework developed to equip developers with code solutions that are more secure than the code initially generated by Large Language Models (LLMs). These code suggestions are obtained through the integration of LLMs and static security code analyzers. LLMSecGuard also offers a feature to measure the security properties of LLMs and provide a current security comparison of different LLMs in the wild.

REFERENCES

[1] Sharmin Afrose, Ya Xiao, Sazzadur Rahaman, Barton P. Miller, and Danfeng Yao. 2023. Evaluation of Static Vulnerability Detection Tools With Java Cryptographic API Benchmarks. *IEEE Transactions on Software Engineering* 49, 2 (2023), 485–497. https://doi.org/10.1109/TSE.2022.3154717

[2] Owura Asare, Meiyappan Nagappan, and N. Asokan. 2023. Is GitHub's Copilot as bad as humans at introducing vulnerabilities in code? *Empirical Software Engineering* 28, 6 (23 Sep 2023), 129. https://doi.org/10.1007/s10664-023-10380-1

[3] Manish Bhatt, Sahana Chennabasappa, Cyrus Nikolaidis, Shengye Wan, Ivan Evtimov, Dominik Gabi, Daniel Song, Faizan Ahmad, Cornelius Aschermann, Lorenzo Fontana, et al. 2023. Purple Llama CyberSecEval: A Secure Coding Benchmark for Language Models. *arXiv preprint arXiv:2312.04724* (2023).

[4] Noah Bühlmann and Mohammad Ghafari. 2022. How Do Developers Deal with Security Issue Reports on GitHub?. In *Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing* (Virtual Event) *(SAC '22)*.

[5] Claudio Corrodi, Timo Spring, Mohammad Ghafari, and Oscar Nierstrasz. 2018. Idea: Benchmarking Android Data Leak Detection Tools. In *Engineering Secure Software and Systems: 10th International Symposium* (Paris, France) *(ESSoS '18)*. 116–123. https://doi.org/10.1007/978-3-319-94496-8_9

[6] Matteo Esposito, Valentina Falaschi, and Davide Falessi. 2024. An Extensive Comparison of Static Application Security Testing Tools. In *28th International Conference on Evaluation and Assessment in Software Engineering* (Salerno, Italy) *(EASE '24)*.

[7] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large Language Models for Software Engineering: Survey and Open Problems. arXiv:2310.03533

[8] Yujia Fu, Peng Liang, Amjed Tahir, Zengyang Li, Mojtaba Shahin, and Jiaxin Yu. 2023. Security Weaknesses of Copilot Generated Code in GitHub. *arXiv preprint arXiv:2310.02059* (2023).

[9] Pascal Gadient, Mohammad Ghafari, Patrick Frischknecht, and Oscar Nierstrasz. 2019. Security code smells in Android ICC. *Empirical Software Engineering* 24, 5 (01 Oct 2019), 3046–3076. https://doi.org/10.1007/s10664-018-9673-y

[10] Pascal Gadient, Marc-Andrea Tarnutzer, Oscar Nierstrasz, and Mohammad Ghafari. 2021. Security Smells Pervade Mobile App Servers. In *Proceedings of the 15th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)* (Bari, Italy) *(ESEM '21)*. https://doi.org/10.1145/3475716.3475780

[11] M. Ghafari, P. Gadient, and O. Nierstrasz. 2017. Security Smells in Android. In *2017 IEEE 17th International Working Conference on Source Code Analysis and Manipulation (SCAM)*. 121–130. https://doi.org/10.1109/SCAM.2017.24

[12] GitHub. 2023. *GitHub Copilot for Business is Now Available*. https://github.blog/2023-02-14-github-copilot-for-business-is-now-available/ Accessed on January 28, 2024.

[13] Hossein Hajipour, Keno Hassler, Thorsten Holz, Lea Schönherr, and Mario Fritz. 2024. CodeLMSec Benchmark: Systematically Evaluating and Finding Security Vulnerabilities in Black-Box Code Language Models. In *2nd IEEE Conference on Secure and Trustworthy Machine Learning* (Toronto, Canada) *(SaTML '24)*.

[14] M. Hazhirpasand and M. Ghafari. 2021. Worrisome Patterns in Developers: A Survey in Cryptography. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*. 185–190. https://doi.org/10.1109/ASEW52652.2021.00045

[15] M. Hazhirpasand, M. Ghafari, S. Kruger, E. Bodden, and O. Nierstrasz. 2019. The Impact of Developer Experience in Using Java Cryptography. In *2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*.

[16] Mohammadreza Hazhirpasand, Mohammad Ghafari, and Oscar Nierstrasz. 2020. Java Cryptography Uses in the Wild. In *Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*.

[17] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2023. Large Language Models for Software Engineering: A Systematic Literature Review. arXiv:2308.10620 [cs.SE]

[18] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. 2023. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. arXiv:2311.05232 [cs.CL]

[19] Meta AI. Year of publication or last update. *LLAMA: Language Model for Many Applications*. https://ai.meta.com/llama/ Accessed on January 28, 2024.

[20] Zahra Mousavi, Chadni Islam, Kristen Moore, Alsharif Abuadbba, and Muhammad Ali Babar. 2024. An Investigation into Misuse of Java Security APIs by Large Language Models.

[21] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the keyboard? assessing the security of github copilot's code contributions. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 754–768.

[22] Semgrep. Year of the latest commit or release. Semgrep: Lightweight static analysis for many languages. https://github.com/semgrep/semgrep. Accessed on January 28, 2024.

[23] Quentin Stiévenart, Coen De Roover, and Mohammad Ghafari. 2022. Security Risks of Porting C Programs to Webassembly. In *Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing* (Virtual Event) *(SAC '22)*. Association for Computing Machinery.

[24] Yuriy Tymchuk, Mohammad Ghafari, and Oscar Nierstrasz. 2018. JIT feedback: what experienced developers like about static analysis. In *Proceedings of the 26th Conference on Program Comprehension* (Gothenburg, Sweden) *(ICPC '18)*. 64–73.

[25] Zhilong Wang, Lan Zhang, Chen Cao, and Peng Liu. 2023. The Effectiveness of Large Language Models (ChatGPT and CodeBERT) for Security-Oriented Code Analysis. (2023). https://doi.org/10.2139/ssrn.4567887

[26] Weggli-RS. Year of the latest commit or release. Weggli: A Rust implementation of the Wegman-Carter Universal Hashing scheme. https://github.com/weggli-rs/weggli. Accessed on January 28, 2024.

[27] Jos Wetzels, Daniel Dos Santos, and Mohammad Ghafari. 2023. Insecure by Design in the Backbone of Critical Infrastructure. In *Proceedings of Cyber-Physical Systems and Internet of Things Week 2023* (San Antonio, TX, USA) *(CPS-IoT Week '23)*. Association for Computing Machinery.

[28] Ying Zhang, Md Mahir Asef Kabir, Ya Xiao, Danfeng Yao, and Na Meng. 2023. Automatic Detection of Java Cryptographic API Misuses: Are We There Yet? *IEEE Transactions on Software Engineering* 49, 1 (2023), 288–303.